

Helix Thready — TUI Usage Guide

Table of contents

1. Launching & connecting

2. Screen layout (diagram)

3. Navigation & keybindings

4. Views

5. The live event stream

6. Accessibility & theming

7. Tutorials

8. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (user-guides)	Initial Bubble Tea TUI guide
2	2026-07-22	swarm (user-guides, Pass 3)	Depth pass: split the screen-layout diagram explanation into multi-paragraph form; added VERIFIED helix_track_cli facts (Bubble Tea + Lip Gloss, OpenDesign theme alignment, ~/.config/<app>/config.yaml, FOUNDATION/design-first); expanded the keybinding reference and tutorials

The Thready TUI is a full-screen terminal application built with **Bubble Tea + Cobra + Lipgloss** (the helix_track_cli pattern) [IN-HOUSE]. “Everything possible from the Web works from the TUI” (final request § TUI). It uses the **same REST /v1 + event bus** as every client, so it is fully interactive and live. Launch with `thready tui`.

VERIFIED vs ASSUMPTION. The framework (Bubble Tea + Cobra + Lipgloss) is VERIFIED from the decision matrix. The pane layout and keybindings below are this guide’s proposal [DEFAULT – adjustable]. [GAP register §8.6] helix_track_cli is FOUNDATION and not deep-audited — verify the exact widgets before finalizing.

Table of contents

1. Launching & connecting
2. Screen layout (diagram)
3. Navigation & keybindings
4. Views
5. The live event stream
6. Accessibility & theming
7. Tutorials
8. Open items

1. Launching & connecting

```
thready auth login          # once; session stored in OS keyring
thready tui                 # launches the full-screen UI
thready tui --account Acme  # start scoped to an account
```

The TUI reuses your CLI session and server config ([cli-reference.md §1](#)). If not authenticated it drops you into an inline login form.

2. Screen layout (diagram)

thready tui — main screen

Header: brand · account ·
connection · MFA state



Body (three panes)

Left nav
Channels ►
Threads
Search
Assets
Events
Admin*

Center list
selected view's rows
(threads / results / assets)

Right detail
post / asset / status reply
+ live processing state

Made with love ❤️ by Helix Development

Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: [diagrams/tui-layout.mmd](#).

Explanation (for readers/models that cannot see the diagram). The TUI is a single full-screen layout with four horizontal bands, stacked top to bottom. This four-band structure is what makes the terminal client feel like a dashboard rather than a REPL: persistent context at the top, the working area in the middle, a live feed near the bottom, and help at the very bottom.

The **header** shows the active brand (respecting per-Account white-label), the selected Account, connection health to the API/event bus, and your MFA/auth state. It is always visible so you never lose track of *which* account you are acting on or whether the live connection is healthy — a dropped event-bus link shows here before you notice stale data.

The **body** is split into three vertical panes that form a classic master-detail flow. The **left navigation** lists the views (Channels, Threads, Search, Assets, Events, and an Admin section marked  that only appears for Admin tiers). The **center list** shows the rows for the selected view — threads, search results, or assets. The **right detail** pane renders the selected item — a complete post with its status reply and asset links, or an asset's metadata — including its **live processing state** that updates as events arrive. Focus flows left→center→right as you drill in.

Across the bottom sits the **live event stream**, a scrolling log of `post.received` / `post.processed` / error events pushed over WebSocket/SSE, so you watch processing happen in real time without refreshing. Because this band is always on screen, the TUI is the best surface for *watching* a channel process a burst of posts, where the Web portal would need you to keep a feed panel open.

The **footer** shows context-sensitive keybindings and carries the Helix Development slogan. Focus moves between the three body panes and the event stream via `Tab`; the whole layout is the terminal analogue of the **web portal** master-detail screens, which is why a workflow learned in one transfers to the other. The colour theme aligns with the shared **OpenDesign** tokens, matching the VERIFIED `helix_track_cli` reference (Bubble Tea + Lip Gloss).

3. Navigation & keybindings

Key	Action
<code>Tab</code> / <code>Shift+Tab</code>	Move focus between panes
<code>↑</code> / <code>↓</code> or <code>k</code> / <code>j</code>	Move within a list
<code>Enter</code>	Open / drill into the selected item
<code>Esc</code>	Back / close detail
<code>/</code>	Focus search (semantic query)
<code>g</code> then a letter	Jump to view: <code>gc</code> Channels, <code>gt</code> Threads, <code>gs</code> Search, <code>ga</code> Assets, <code>ge</code> Events
<code>r</code>	Re-process selected post (<code>thready post reprocess</code>)
<code>R</code>	Retry a failed step (choose step)
<code>p</code>	Pause/resume processing (Admin; scope prompt)
<code>f</code>	Filter current list (since/kind/status)
<code>y</code>	Copy selected id/asset ref to clipboard
<code>?</code>	Keybinding help overlay
<code>q</code> / <code>Ctrl+C</code>	Quit

4. Views

- **Channels** — list channels, add (invite link), set poll interval, remove. Telegram only for the zero version `[GAP: 3]`; Max shows a “not yet available” badge.
- **Threads** — browse complete posts per channel; the detail pane shows root + organic replies, detected categories, status reply, and linked assets/research.
- **Search** — type a query; results are semantic over posts + generated materials (< 500 ms target). `[GAP: 1]` if results look wrong, the server is on the hash embedder — the header shows an `embedder: hash` ⚠ warning.
- **Assets** — list/get renditions (raw/ `-web`), reheal broken links. Downloads resolve via the Asset Service (never direct paths).
- **Events** — full-screen view of the live stream with filters.
- **Admin** (Admin tiers) — members, skills, retention, branding, processing pause/resume, billing view.

5. The live event stream

The bottom band subscribes to the event bus (WebSocket, SSE fallback). It renders one line per event with a severity color. Durable JetStream consumers replay missed events on reconnect (final request §3.4), so if

your terminal drops the connection you don't lose events — the stream backfills. The full set of topics the stream can show (and their one-time vs sticky semantics) is the consolidated [event catalog](#); the `ge` Events view lets you filter to any subset.

```
09:41:02 post.received chan=devops post=8f3a... #Research #Video
09:41:07 skill.dispatch post=8f3a... order=[download,research,reply]
09:41:55 post.processed post=8f3a... assets=1 research=1 took=48s
09:42:10 download.error post=91b2... step=metube retry=2/5 (backoff 8s)
```

6. Accessibility & theming

- Respects `NO_COLOR` and 16/256/truecolor terminals via Lipgloss.
- Light + dark theming derived from the OpenDesign tokens (helix-green `#B6E376` default), honoring per-Account branding in the header `[CONSTITUTION $11.4.162]`.
- All actions have keyboard equivalents (no mouse required); screen-reader-friendly plain-text mode via `thready tui --plain`.

7. Tutorials

Tutorial A — Watch a post process end-to-end. 1. `thready tui` → `gc` Channels → select a channel → `Enter`. 2. Post a `#Research #Video` message in that Telegram channel from your account. 3. Watch the bottom stream: `post.received` → `skill.dispatch` → `post.processed`. 4. `gt` Threads → open the post → see the status reply and the new asset/research in the detail pane.

Tutorial B — Triage a failed download. 1. `ge` Events → filter `download.error`. 2. Select the post → `R` → choose `download` to retry the step. 3. If it keeps failing, the FAQ/troubleshooting entry for [downloads](#) explains the MeTube poll-only caveat.

8. Open items

- `[OPEN: tui-1]` Pane layout & keymap are `[DEFAULT — adjustable]`; validate against the implemented Bubble Tea models. Tracked: **ATM — finalize TUI keymap + panes.**
- `[OPEN: tui-2]` `helix_track_cli` reference is **VERIFIED FOUNDATION / design-first** — its README states “no functional code yet” and its `docs/CLI_SPEC.md` holds the *planned* Bubble Tea screen map and keybindings. The stack (Bubble Tea + Lip Gloss, OpenDesign theme, `~/.config/<app>/config.yaml`) is confirmed and adopted; the concrete widgets are not yet implemented, so validate the pane models against Thready's own build before freezing. Tracked: **ATM — implement + snapshot-test TUI panes against the frozen keymap.**

Made with love ♥ by Helix Development.